实验报告: LC-3 序列检测器

Name and Date

Name: 黄思睿
Date: 2024年12月23日

The Core Algorithm

目标

通过 LC-3 汇编语言编写一个程序，检测用户输入的二进制序列中 `1010` 序列出现的次数。程序使用有限状态机（FSM）来逐步检测 `1010`，并在输入结束后输出出现次数。

算法设计

1. **输入部分**：
   - 显示提示信息，等待用户输入二进制数字序列。
   - 用户输入的每个字符都会回显到屏幕上，直到输入字符 `y` 为止，表示结束输入。
2. **有限状态机**：
   - 程序使用有限状态机（FSM）来逐个字符判断并识别是否匹配 `1010` 序列。
   - 程序的状态设计如下：
     - **S0**: 初始状态，等待字符 `1`。
     - **S1**: 已检测到 `1`，等待字符 `0`。
     - **S2**: 已检测到 `10`，等待字符 `1`。
     - **S3**: 已检测到 `101`，等待字符 `0`（完成 `1010`，计数加 1）。
3. **状态转换**：
   - 每次检测到完整的 `1010` 序列时，计数器 `R5` 增加 1，回到初始状态 S0 或继续检测下一个可能的 `1010`。
4. **输出部分**：
   - 输出换行符后，显示结果消息和检测到的 `1010` 序列的出现次数。

Critical Code to Implement the Algorithm

以下是实现该算法的关键代码片段。

1. 输入循环与回显

```
; 输入循环
LD R2, BUF_PTR_INIT        ; 初始化输入缓冲区指针
LD R3, LOWER               ; R3 = -48, 让输入的 ASCII 码变成 对应数字
LD R4, Y                   ; R4 检测 y
INPUT_LOOP:
    GETC                   ; 从键盘获取一个字符
    ADD R1, R0, R3         ; 将字符复制到 R1,  并转换为 数字
    STR R1, R2, #0         ; 将数字存储到缓冲区
    ADD R2, R2, #1         ; 增加缓冲区指针
    PUTC
    ADD R1, R1, R4         ; 检查字符是否为 'y' (ASCII 89)
    BRz INPUT_END          ; 如果是 'y', 结束输入
    BR INPUT_LOOP          ; 否则继续输入

INPUT_END
```

- 关键点：
  - 每次从键盘获取一个字符并转换为二进制数字。
  - 使用 `PUTC` 回显字符，直到输入 `y` 为止。

2. 有限状态机 (FSM)

2.1 状态机部分对应代码：

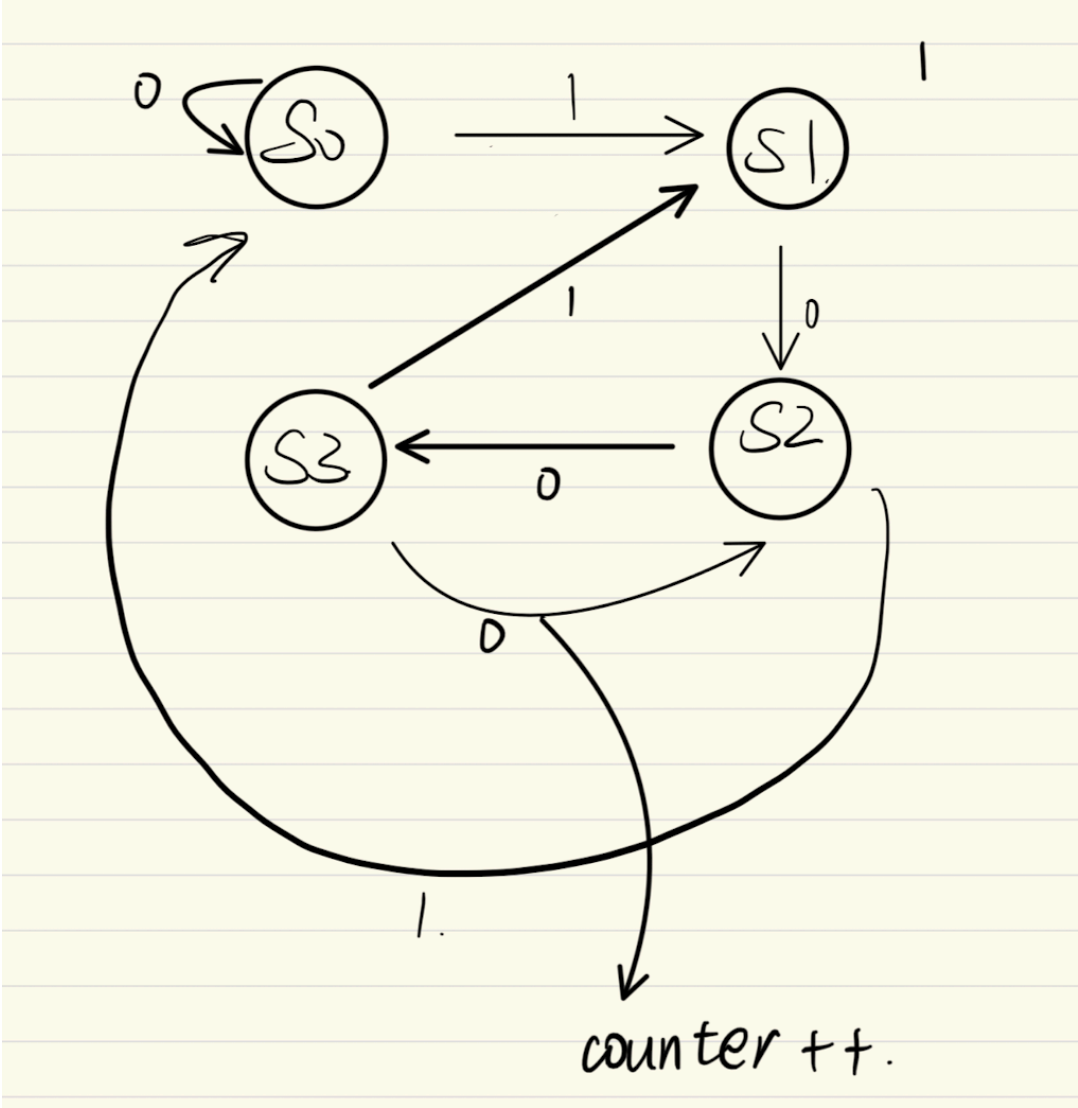
```
; 有限状态机部分, 用于统计 "1010" 的出现次数
LD R2, BUF_PTR_INIT        ; 重置缓冲区指针
AND R3, R3, #0            ; 清空状态寄存器
AND R5, R5, #0            ; 清空计数器

FSM_LOOP:
    LDR R0, R2, #0        ; 从缓冲区加载下一个字符
    ADD R2, R2, #1        ; 移动到下一个字符
    ADD R1, R0, R4
    BRz FSM_END           ; 如果已经遍历完成, 结束有限状态机

STATE_0
    ; 状态转换
    ; 状态 0: 初始状态, 寻找 '1'
    ADD R1, R3, #0        ; COPY R3 到 R1
    ADD R1, R1, #0        ; 比较当前状态是否为 0
    BRnp STATE_1          ; 如果不是状态 0, 跳过
    ADD R3, R0, #0        ; 如果输入是 '1', 移动到状态 1
    BR FSM_LOOP           ; 继续

STATE_1:
```

```
    ADD R1, R3, #0          ; 将当前状态（R3）复制到 R1
    ADD R1, R1, #-1         ; 比较当前状态是否为 1（R3 - 1）
    BRnp STATE_2             ; 如果当前状态是 1，跳转到 STATE_2

    ADD R0, R0, #0          ; 读取输入字符
    BRz UPDATE_STATE_2      ; 如果输入为 0，更新状态为 2
    ; 如果输入不是 0，继续保持在 STATE_1
    BR FSM_LOOP             ; 返回到 FSM 主循环

STATE_2:
    ADD R1, R3, #0
    ADD R1, R1, #-2       ; 比较当前状态是否为 2
    BRnp STATE_3          ; 如果不是状态 2，跳过

    ADD R0, R0, #0          ; 读取输入字符
    BRz UPDATE_STATE_0      ; 如果输入为 0，更新状态为 0
    BR UPDATE_STATE_3       ; 如果输入是 1，更新为状态 3


STATE_3:
    ADD R1, R3, #0
    ADD R1, R1, #-3       ; 比较当前状态是否为 3
    BRnp FSM_LOOP         ; 如果不是状态 3，继续

    ADD R0, R0, #0          ; 读取输入字符
    BRz COUNTER_UP           ; 如果输入为 0，更新状态为 2，count++
    BR UPDATE_STATE_1       ; 如果输入是 1，更新为状态 1

UPDATE_STATE_0:
    AND R3, R3, #0
    BR FSM_LOOP             ; 继续返回到 FSM 主循环

UPDATE_STATE_1:
    AND R3, R3, #0
    ADD R3, R3, #1          ; 更新状态为 1（即 R3 = 1）
    BR FSM_LOOP             ; 继续返回到 FSM 主循环

UPDATE_STATE_2:
    AND R3, R3, #0
    ADD R3, R3, #2          ; 更新状态为 2（即 R3 = 2）
    BR FSM_LOOP             ; 继续返回到 FSM 主循环

UPDATE_STATE_3:
    AND R3, R3, #0
    ADD R3, R3, #3          ; 更新状态为 3（即 R3 = 3）
    BR FSM_LOOP             ; 继续返回到 FSM 主循环

COUNTER_UP
    AND R3, R3, #0
```

```
    ADD R3, R3, #2          ; 更新状态为 2（即 R3 = 2）
    ADD R5, R5, #1         ; 计数器加 1
    BR FSM_LOOP            ; 返回有限状态机循环

FSM_END:
    LD R4, UPPER
    ADD R5, R4, R5
    STI R5, COUNT          ; 存储计数器
```

- 关键点：
  - 状态转换逻辑判断字符序列，逐步检测 `1010`。
  - 在每个状态下判断当前输入并根据状态更新。

2.2 对应状态图：



3. 输出计数结果

```
; 显示结果
LD R0, NEWLINE
PUTC                        ; 输出换行符
LEA R0, RESULT_MSG          ; 加载结果消息
PUTS
LDI R0, COUNT               ; 加载计数
OUT                         ; 输出计数
LEA R0, RESULT_COUNT        ; 加载结果后缀
PUTS
HALT
```

- 关键点：
  - 输出换行符后，显示统计结果和 `1010` 出现的次数。

4.全部代码：

```
.ORIG x3000
; 初始化部分
LEA R0, PROMPT              ; 加载提示信息的地址
PUTS                        ; 显示提示信息

; 输入循环
LD R2, BUF_PTR_INIT         ; 初始化输入缓冲区指针
LD R3, LOWER                ; R3 = -48, 让输入的 ASCII 码变成 对应数字
LD R4, Y                    ; R4 检测 y
INPUT_LOOP:
    GETC                    ; 从键盘获取一个字符
    ADD R1, R0, R3          ; 将字符复制到 R1,  并转换为 数字
    STR R1, R2, #0          ; 将数字存储到缓冲区
    ADD R2, R2, #1          ; 增加缓冲区指针
    PUTC
    ADD R1, R1, R4          ; 检查字符是否为 'y' (ASCII 89)
    BRz INPUT_END           ; 如果是 'y', 结束输入
    BR INPUT_LOOP           ; 否则继续输入

INPUT_END
; 有限状态机部分，用于统计 "1010" 的出现次数
LD R2, BUF_PTR_INIT         ; 重置缓冲区指针
AND R3, R3, #0              ; 清空状态寄存器
AND R5, R5, #0              ; 清空计数器

FSM_LOOP:
    LDR R0, R2, #0          ; 从缓冲区加载下一个字符
    ADD R2, R2, #1          ; 移动到下一个字符
    ADD R1, R0, R4
```

```
    BRz FSM_END           ; 如果已经遍历完成，结束有限状态机

STATE_0
    ; 状态转换
    ; 状态 0：初始状态，寻找 '1'
    ADD R1, R3, #0        ; COPY R3 到 R1
    ADD R1, R1, #0        ; 比较当前状态是否为 0
    BRnp STATE_1          ; 如果不是状态 0，跳过
    ADD R3, R0, #0        ; 如果输入是 '1'，移动到状态 1
    BR FSM_LOOP           ; 继续

STATE_1:
    ADD R1, R3, #0          ; 将当前状态（R3）复制到 R1
    ADD R1, R1, #-1         ; 比较当前状态是否为 1（R3 - 1）
    BRnp STATE_2             ; 如果当前状态是 1，跳转到 STATE_2

    ADD R0, R0, #0          ; 读取输入字符
    BRz UPDATE_STATE_2      ; 如果输入为 0，更新状态为 2
    ; 如果输入不是 0，继续保持在 STATE_1
    BR FSM_LOOP             ; 返回到 FSM 主循环

STATE_2:
    ADD R1, R3, #0
    ADD R1, R1, #-2       ; 比较当前状态是否为 2
    BRnp STATE_3          ; 如果不是状态 2，跳过

    ADD R0, R0, #0          ; 读取输入字符
    BRz UPDATE_STATE_0      ; 如果输入为 0，更新状态为 0
    BR UPDATE_STATE_3       ; 如果输入是 1，更新为状态 3


STATE_3:
    ADD R1, R3, #0
    ADD R1, R1, #-3       ; 比较当前状态是否为 3
    BRnp FSM_LOOP         ; 如果不是状态 3，继续

    ADD R0, R0, #0          ; 读取输入字符
    BRz COUNTER_UP           ; 如果输入为 0，更新状态为 2，count++
    BR UPDATE_STATE_1       ; 如果输入是 1，更新为状态 1

UPDATE_STATE_0:
    AND R3, R3, #0
    BR FSM_LOOP             ; 继续返回到 FSM 主循环

UPDATE_STATE_1:
    AND R3, R3, #0
    ADD R3, R3, #1          ; 更新状态为 1（即 R3 = 1）
    BR FSM_LOOP             ; 继续返回到 FSM 主循环
```

```
UPDATE_STATE_2:
    AND R3, R3, #0
    ADD R3, R3, #2          ; 更新状态为 2（即 R3 = 2）
    BR FSM_LOOP             ; 继续返回到 FSM 主循环

UPDATE_STATE_3:
    AND R3, R3, #0
    ADD R3, R3, #3          ; 更新状态为 3（即 R3 = 3）
    BR FSM_LOOP             ; 继续返回到 FSM 主循环

COUNTER_UP
    AND R3, R3, #0
    ADD R3, R3, #2          ; 更新状态为 2（即 R3 = 2）
    ADD R5, R5, #1          ; 计数器加 1
    BR FSM_LOOP             ; 返回有限状态机循环

FSM_END:
    LD R4, UPPER
    ADD R5, R4, R5
    STI R5, COUNT           ; 存储计数器

; 显示结果
LD R0, NEWLINE
PUTC
LEA R0, RESULT_MSG          ; 加载结果消息
PUTS
LDI R0, COUNT               ; 加载计数
OUT                         ; 将计数显示为字符
LEA R0, RESULT_COUNT        ; 加载结果后缀
PUTS
HALT

UPPER        .FILL #48
LOWER        .FILL #-48
Y            .FILL #-73  ; y (ASCII 121) = 48 + 73
BUF_PTR_INIT .FILL x6000 ; 输入缓冲区起始地址
COUNT .FILL x5FFF            ; "1010" 序列的计数器
; 提示信息
PROMPT .STRINGZ "SD is ready! Please input your number:\n"
RESULT_MSG .STRINGZ "There are(is) "
RESULT_COUNT .STRINGZ "  1010 in the sequence!\n"
NEWLINE .FILL x0A  ; 换行符的 ASCII 值

.END
```

Debug Process

1. **输入缓冲区的管理**：

- 在开始输入时，需要清楚地管理缓冲区指针，以确保输入数据能够正确存储并传递到后续部分。
- 通过调试和观察程序执行，我确保了缓冲区指针 `R2` 每次递增正确，避免了缓冲区溢出或数据丢失。

2. **状态机的跳转逻辑**：
   - 状态机的跳转逻辑在初始阶段有些复杂，特别是在使用 `BRnp` 和 `BRz` 进行状态判断时。经过多次调试，确认了在比较当前状态与目标状态时，使用 `BRz` 比较更为直接和简洁。
3. **计数器更新**：
   - 计数器 `R5` 更新的部分一开始没有用`STI`正确存储到目标地址，因而无法显示结果。通过调试，确认了错误成功将结果存储到内存。
4. **输出问题**：
   - 输出时，R5存储的内容应该 `+48` 变为数字对应的 ASCII 码值，否则无法正常输出显示。

Run Results

输入输出测试1：

```
SD is ready! Please input your number:
101010y
There are(is) 2  1010 in the sequence!
```

- 程序成功检测到输入序列中的 `2` 次 `1010`。

输入输出测试2：

```
SD is ready! Please input your number:
10111010111011010y
There are(is) 2  1010 in the sequence!
```

- 程序成功检测到输入序列中的 `2` 次 `1010`。

输入输出测试3:

```
SD is ready! Please input your number:
11010110y
There are(is) 1  1010 in the sequence!
```

- 程序成功检测到输入序列中的 `1` 次 `1010`。

Traps and Pitfalls You've Found

1. **ASCII 字符的处理**：
   - 在输入部分，我需要确保将输入的 ASCII 字符（如 '1' 或 '0'）转换为数字，这需要正确调整字符的 ASCII 值。例如，将 `GETC` 读取的字符通过减去 `48` 转换为二进制数字。
2. **状态机设计的复杂性**：

- 使用有限状态机（FSM）时，设计合理的状态转移非常关键。调试时，我发现一些状态跳转逻辑可能会导致状态重复或跳转不准确。通过细化每个状态的条件判断，最终成功修复了这个问题。

3. **计数器的显示**：
   - 在输出计数时，我需要确保计数器的值正确显示。最初使用 `OUT` 显示数字时遇到了问题，因为 `OUT` 只能输出 ASCII 字符，需要确保计数转换为正确的 ASCII 码。

Suggestions

1. **输入界面改进**：
   - 可以在输入部分添加更多的提示，提醒用户输入的格式要求（如只输入 `0` 或 `1`），以避免错误输入。
2. **错误处理**：
   - 当前的程序没有处理异常输入或错误输入的情况（如输入非 `0` 和 `1` 的字符）。可以考虑添加输入验证部分，提示用户重新输入。